

Polling

Polling is a communication technique where the **client repeatedly checks the server** at fixed intervals to ask whether new data or messages are available.

Instead of the server pushing updates automatically, the client continuously sends requests like:

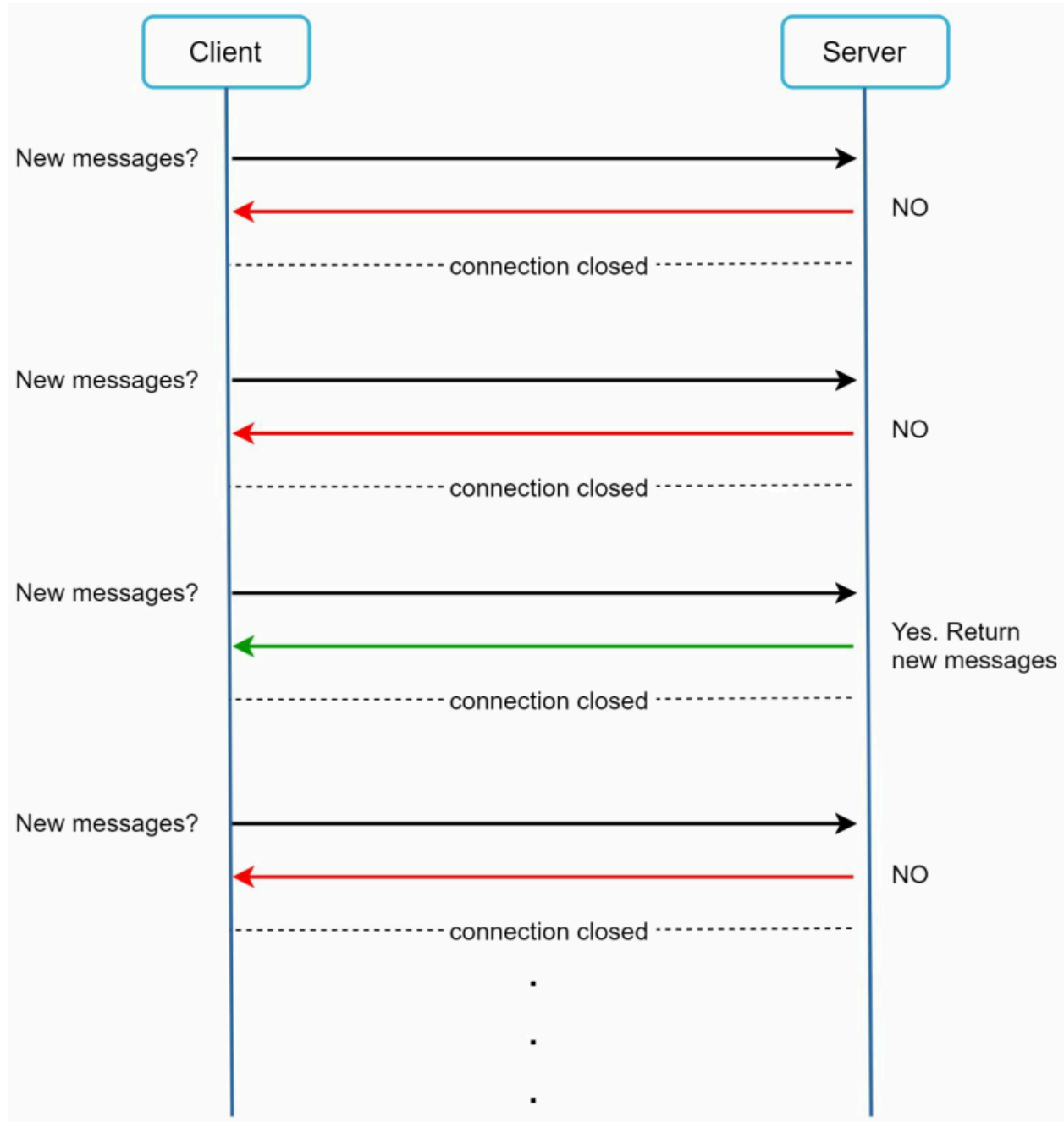
None

Any new messages?

Any notifications?

Any updates?

This concept is commonly discussed in **chat systems, notification systems, real-time dashboards, and distributed systems.**



As shown in **Figure**, polling works by making the client periodically query the server for updates.

1. How Polling Works

Basic flow:

None

1. Client sends request to server
2. Server checks if data is available
3. If yes → send response
4. If no → send empty response
5. Client waits fixed interval
6. Repeat again

Example:

None

Every 5 seconds:

Client → GET /messages

Server → No new messages

Then after 5 seconds:

None

Client → GET /messages

Server → 2 new messages

2. Diagram

None

Time →

Client ----Request----> Server

<----No Data----

(wait 5 sec)

Client ----Request----> Server

<----No Data----

(wait 5 sec)

Client ----Request----> Server

<----New Message---

3. Types of Polling

Short Polling

Client sends requests at regular intervals.

None

Every 2 sec

Every 5 sec

Every 10 sec

Most common form of polling.

Long Polling

Client sends request and server **holds connection open** until new data is available or timeout occurs.

None

Client requests updates

Server waits...

If data arrives → respond immediately

Reduces unnecessary repeated requests.

4. Advantages of Polling

✓ Easy to Implement

Works with normal HTTP requests.

None

GET /notifications

No complex protocol needed.

✓ Firewall Friendly

Uses standard HTTP/HTTPS ports.

✓ Widely Supported

Works in browsers, mobile apps, backend systems.

5. Disadvantages of Polling

Waste of Resources

Most requests return:

None

No updates

Still consumes:

- CPU
- Network bandwidth
- DB reads
- Load balancer resources

Higher Latency

If polling every 10 sec:

None

Message may wait up to 10 sec before user sees it

Poor Scalability

For millions of users:

None

1M users polling every 5 sec

= 200,000 requests/sec

Huge infrastructure load.

6. Example Calculation

Suppose:

None

10 million users

Each polls every 10 sec

Requests per second:

None

$10,000,000 / 10 = 1,000,000$ requests/sec

Even if no messages exist.

This is why polling becomes expensive at scale.

7. Polling in Chat Systems

Bad approach:

None

User checks every 2 sec for messages

Problems:

- High server cost
- Delayed message delivery
- Wasteful empty responses

Better solutions:

None

Long Polling

WebSockets

Server-Sent Events (SSE)

Push Notifications

8. Polling vs Push Model

Feature	Polling	Push/WebSocket
Client Requests Repeatedly	Yes	No
Real-time Speed	Medium	High
Server Load	High	Lower
Easy to Implement	Yes	Moderate

Best for Scale	No	Yes
----------------	----	-----

9. System Design Interview Insight

If interviewer asks:

Why not use polling for chat app?

Answer:

None

Polling generates many unnecessary requests,
increases server load, wastes bandwidth,
and introduces delay in message delivery.

WebSockets are better for real-time communication.

10. Where Polling is Acceptable

Polling is fine for:

None

Low traffic admin dashboards

Background status checks

Small internal tools

Periodic analytics refresh

Rare update systems

11. Modern Alternatives

Long Polling

Less wasteful than short polling.

WebSocket

Persistent bidirectional connection.

SSE

Server continuously streams updates.

Push Notifications

Mobile/browser alerts.

12. One-Line Summary

Polling is a simple client-driven update mechanism where clients repeatedly ask the server for new data, but it wastes resources and scales poorly for real-time systems.